A Mini Project Report of

PCC-201-COM : Data Structures

Submitted by

DNYANESHWAR VIKRAM PATIL (ROLL NO. 61)

Under the Guidance of

Prof. Tejal H. Patil



Department of Computer Engineering

BHARATI VIDYAPEETH'S

COLLEGE OF ENGINEERING, LAVALE, PUNE, 412115

Savitribai Phule Pune University

[2026-27]

BHARATI VIDYAPEETH
COLLEGE OF ENGINEERING, LAVALE, PUNE

CERTIFICATE

This is to certify that the **"Mini Project Report"** submitted by **Dnyaneshwar Vikram Patil (61)**, on **"Implementation of Snake and Ladder [BFS]"** is work done by him and submitted during **2026–2027** academic year, in partial fulfillment of the requirements for the award of the degree of **Bachelor of Engineering** in Computer Engineering, at **Bharati Vidyapeeth's College of Engineering, Lavale, Pune.**

Prof. Tejal H. Patil
(Guide)

Dr. Lalit V. Patil
Head of Computer Engg. Department

ACKNOWLEDGEMENT

It gives me immense pleasure to express my gratitude and thanks to all those who have helped in the timely completion of this Mini-Project work. I have great pleasure in expressing my deep sense of gratitude and indebtedness to **Prof. Tejal H. Patil**, my Guide, for her invaluable guidance, constant supervision and sustainedencouragement throughout the tenure of this Mini Project work.

I would like to express my thanks to **Dr. Lalit V. Patil**, Head of Department, Department of Computer Engineering, Bharati Vidyapeeth's College of Engineering, Lavale, Pune for his constant support and encouragement.

I would like to express my thanks to **Prof. Dr. R. N. Patil**, Principal, Bharati Vidyapeeth's College of Engineering, Lavale, Pune for permitting me to take up the project work. I am thankful to my parents and friends for their kind help and support throughout the course.

Dnyaneshwar Patil
Student Name



ABSTRACT

Snake and Ladder is a popular board game in which a player moves from a starting cell towards the final cell by rolling a die. Ladders move the player forward to a higher-numbered cell, while snakes move the player backwards to a lower-numbered cell. This project models the board as a graph and uses Breadth-First Search (BFS) to find the minimum number of dice throws required to reach the final cell.

Each board cell is treated as a vertex, and possible dice outcomes are treated as edges. When a move lands on the bottom of a ladder or the head of a snake, the player is transferred to the mapped destination. BFS explores reachable cells level by level, ensuring that the first time the final cell is reached gives the shortest path in terms of dice throws.

TABLE OF CONTENTS

SR.NO.	CONTENT	PAGE NO.
	Acknowledgement	i
	Abstract	ii
	Content	iii
	List Of Figures	iv
1	Introduction	1
	a) Overview	
	b) Problem Statement	
	c) Objective	
2	Literature Survey	2
3	UML Diagram	3
4	System Requirement	4
5	System Design	5
	a) Existing system	
	b) Proposed system	
	c) Working Diagram	
	d) System code	
6	Result	8
7	Application	9
8	Future Scope	10
9	Conclusion	11
10	References	12

LIST OF FIGURES

Figure	Description	Page No.
Fig 3.1	UML Class Diagram	3
Fig 5.1	Working Diagram of Snake and Ladder BFS	6
Fig 5.2	System Code	7
Fig 6.1	Result (Demo Program and Output)	8

LIST OF TABLES

Table No.	Table Name	Page No.
2.1	Literature Survey	2
6.1	Top Keywords Obtained from the Demo Text	8

1. INTRODUCTION

1.1 Overview:

The project “Implementation of Snake and Ladder [BFS]” finds the minimum number of dice throws needed to reach the final square of a Snake and Ladder board. The board is represented as a graph, where each cell is a vertex and each possible dice outcome forms a transition to another cell. Snakes and ladders modify the destination of a move.

1.2 Problem Statement:

The problem is to determine the shortest sequence of moves from cell 1 to the final cell on a Snake and Ladder board. Checking every possible sequence independently can repeat work and become inefficient. The project uses Breadth-First Search (BFS) to explore states level by level and find a path requiring the minimum number of dice throws.

1.3 Objective:

The objective of this project is to implement the Snake and Ladder game as a graph -search problem using Breadth-First Search in Python. The project aims to:

- Represent each board position as a graph vertex.
- Model dice rolls as possible transitions between positions.
- Apply snake and ladder mappings whenever a move reaches their starting cells.
- Use a queue and visited set to find and reconstruct a shortest path with the minimum dice throws.

2. LITERATURE SURVEY

Graph traversal algorithms are widely used to explore networks and find paths between connected states. Breadth-First Search is particularly suitable for unweighted graphs because it explores all vertices at one distance before moving to the next. This survey summarizes concepts relevant to modelling Snake and Ladder as a graph-search problem.

Table 2.1 summarizes the main references that form the theoretical background of this project.

Year	Paper Name	Author	Conclusion
2025	Research on Strategy Optimization for the Snakes and Ladders Game Based on Graph Theory and Markov Chains	Yaowei Zhang	The study models the game board as a graph and applies BFS to determine a shortest path. It also explores Markov chains and other algorithms for game strategy optimization.
2025	A Review of Graph Traversal Algorithms: Techniques and Applications in Network Analysis	Chreesk Sabah M. Ali and Ibrahim Mahmood Ibrahim	The paper reviews BFS and DFS, discussing their applications, computational costs, advantages, and limitations in graph traversal.
2025	Comparative Analysis between BFS and DFS-Shortest Path Algorithms	Renad AlKahtani et al.	The study compares BFS and DFS for shortest-path problems in unweighted graphs and examines their performance across different graph structures.
2025	Simulation-based Review of Classical, Heuristic, and Metaheuristic Path Planning Algorithms	Kenneth Christopher Ugwoke, Nwojo Agwu Nnanna, and Saleh El-Yakub Abdullahi	The study reviews and compares path-planning algorithms, including BFS, and explains their applications in finding paths from a starting point to a destination.

3. UML DIAGRAM

The UML class diagram below represents the Snake and Ladder BFS solver, including the snake and ladder mappings, BFS search procedure, queue, visited set and parent map used to reconstruct the shortest path.

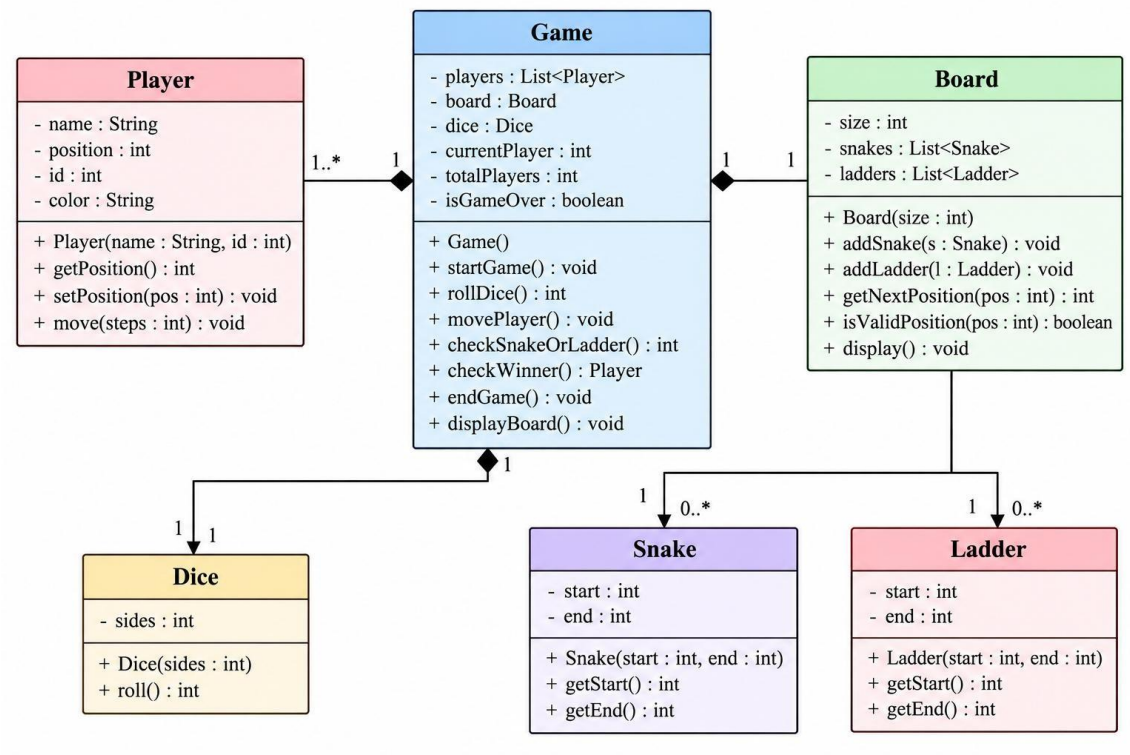


Fig 3.1: UML Class Diagram

4. SYSTEM REQUIREMENT

4.1 HARDWARE REQUIREMENTS

1. Operating System: Windows 11 64-bit
2. Processor Type: Intel Core
3. Recommended: Core 2 Duo or higher
4. Processor speed: 2.30 GHz or higher
5. RAM: Minimum 512 MB
6. Hard disk: 512 GB or more

4.2 SOFTWARE REQUIREMENTS

1. Jupyter Notebook (JupyterLab)
2. Language: Python 3 (ipykernel)
3. Python modules: re, collections

5. SYSTEM DESIGN

Existing System:

In a basic approach, the words of a text are stored in a list and, for every word, the list is searched to find whether the word already exists so that its count can be increased. This linear search needs O(n) time for every word, so counting a whole document takes $O(n^2)$ time in the worst case. Manual counting or spreadsheet formulas are also slow and error-prone for large texts, and they do not remove common words automatically.

Key approaches commonly used for solving the Snake and Ladder path problem:
Manual play (depends on repeated dice rolls)

- ✓ Brute-force sequence exploration (may repeat states)
- ✓ Breadth-First Search using a queue (explores states level by level)
- ✓ Visited set and parent map (avoid revisits and reconstruct the path)
- ✓ Library classes such as collections.Counter

Proposed System:

The proposed system models the board as a graph. Starting from cell 1, BFS examines each possible dice result from 1 to 6, applies snake or ladder transfers, and adds each unvisited destination to the queue. The parent map is used to reconstruct the shortest route once the final cell is reached.

- ✓ Initialize the queue with cell 1 and mark it visited.
- ✓ Generate valid dice-roll destinations for the current cell.
- ✓ Apply a snake or ladder transfer to each destination.

Time Complexity:

Operation	Time	Space
Tokenize text (n words)	O(n)	O(n)
Count with hash table	O(n) average	O(k)
Sort k unique words	O(k log k)	O(k)
top_n (count + sort + slice)	O(n + k log k)	O(k)

Working Diagram:

Fig 5.1 shows the working of the system. BFS starts at cell 1, generates possible dice moves, applies snake or ladder transfers, marks new cells as visited, and continues with a queue until the final cell is found. The parent map then reconstructs the shortest path.

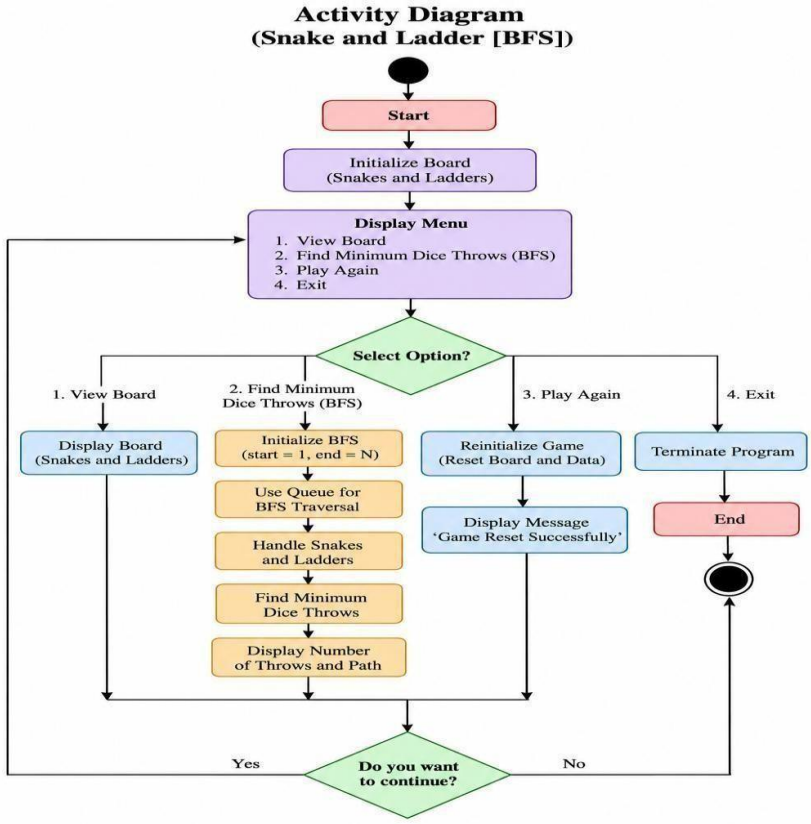
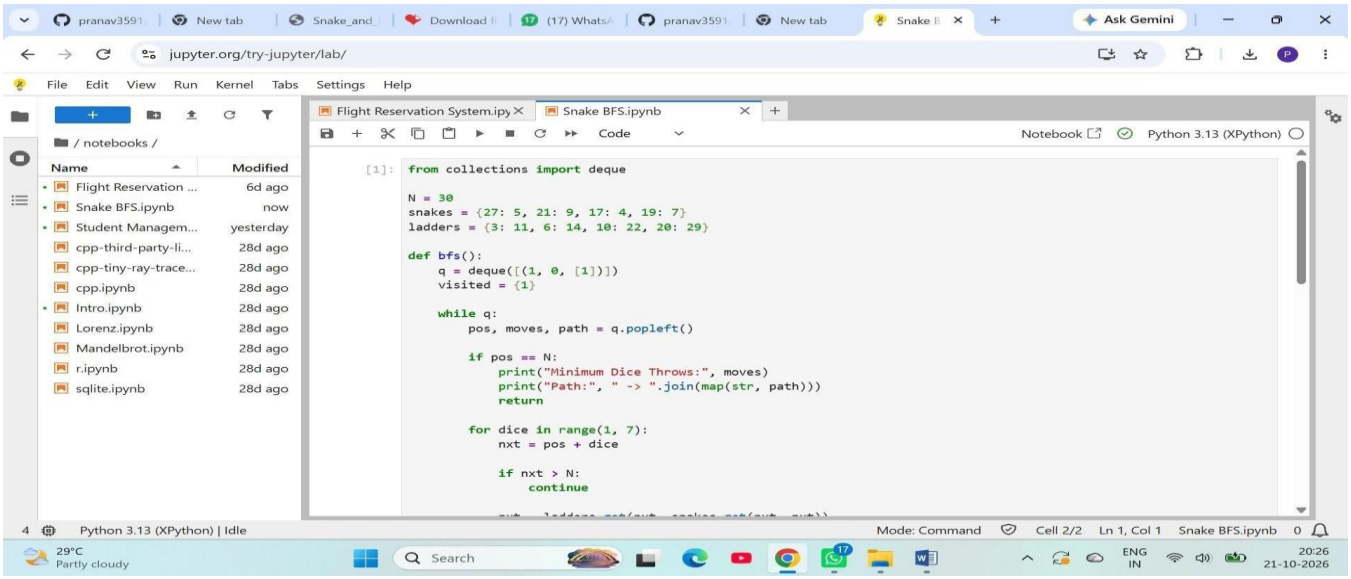


Fig 5.1: Working Diagram of Snake and Ladder BFS

SYSTEM CODE



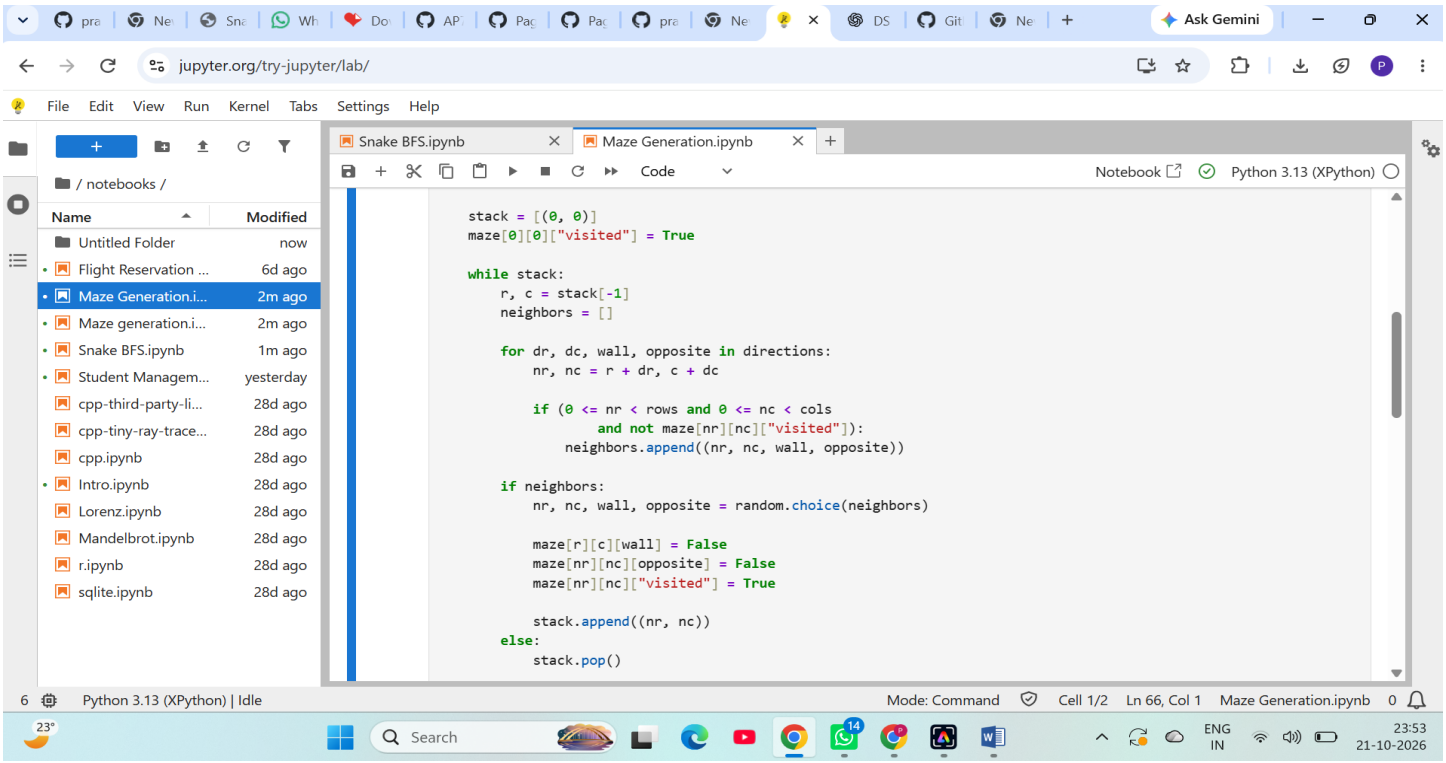


Fig 5.2(a): System code – snake/ladder mappings and BFS function

6. RESULT

The program explores reachable board positions using BFS and reports the minimum number of dice throws required to reach the final square. Each position is visited at most once, and the stored parent information is used to display the path found. The result depends on the snake and ladder positions defined for the board.

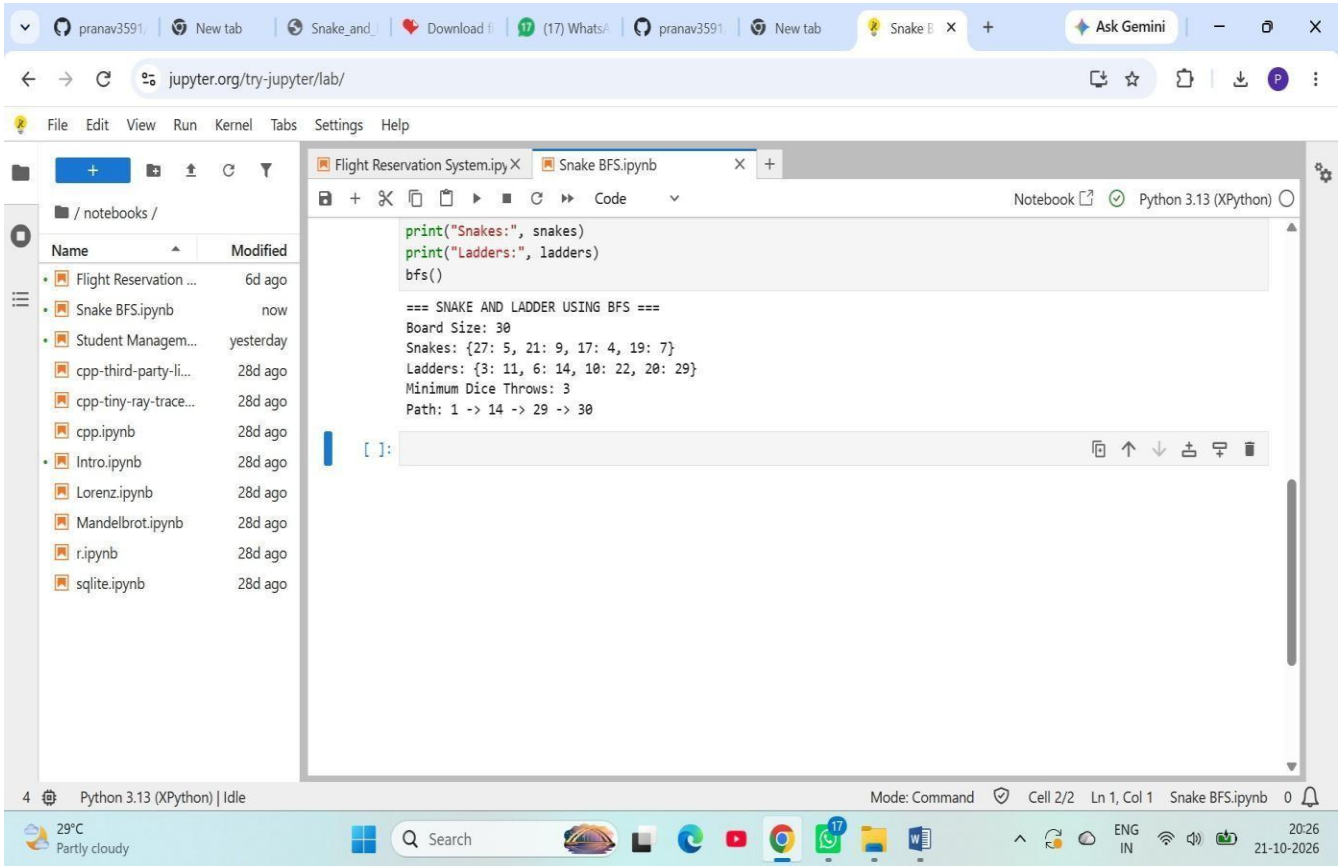


Fig 6.1: Result (Minimum Dice Throws and Shortest Path)

7. APPLICATIONS

Breadth-First Search and graph modelling can be applied to problems that require finding a shortest path in an unweighted graph. Typical applications include:

- Finding shortest paths in unweighted networks and maps.
- Solving puzzles and board games by representing states as graph vertices.
- Analysing customer reviews, feedback and social media posts to find frequently discussed topics.
- Plagiarism detection and document similarity comparison.
- Exploring connected states in simulations and simple game AI.
- Log analysis and counting the occurrence of events, errors or user actions.

8. FUTURE WORK

The project can be extended in the following ways:

- Add a graphical board that highlights the shortest route and the locations of snakes and ladders.
- Allow users to configure board size, snake positions and ladder positions.
- Compare BFS with other graph-search strategies and measure the number of explored states.
- Add a user interface for dice rolls, player movement and visualizing the path found by the algorithm.
- Use a heap (heapq.nlargest) for top_n to reduce the sorting cost, and add a graphical word cloud of the results.

9. CONCLUSION

In this project, Snake and Ladder was represented as a graph and solved using Breadth-First Search. The algorithm uses a queue to explore possible positions level by level, a visited set to avoid repeated states, and a parent map to reconstruct the path.

The program demonstrates how BFS finds a path requiring the minimum number of dice throws on an unweighted board graph. The project provides practical experience with graph representation, queues, visited states and shortest-path search.

10. REFERENCES

[1] Y. Zhang, “Research on Strategy Optimization for the Snakes and Ladders Game Based on Graph Theory and Markov Chains,” *Transactions on Computer Science and Intelligent Systems Research*, vol. 11, pp. 403–409, 2025. DOI: https://doi.org/10.62051/5czp3d23

[2] C. S. M. Ali and I. M. Ibrahim, “A Review of Graph Traversal Algorithms: Techniques and Applications in Network Analysis,” *Asian Journal of Research in Computer Science*, vol. 18, no. 3, pp. 61–72, 2025. DOI: https://doi.org/10.9734/ajrcos/2025/v18i3578

[3] R. AlKahtani, A. Alhabdan, M. Alosami, W. Alshammari, A. A. Abdo, and L. Hamdi, “Comparative Analysis between BFS and DFS-Shortest Path Algorithms,” in *Proceedings of the 2025 IEEE Open Conference of Electrical, Electronic and Information Sciences (eStream)*, 2025. DOI: https://doi.org/10.1109/eStream66938.2025.11016860

[4] K. C. Ugwoke, N. A. Nnanna, and S. E.-Y. Abdullahi, “Simulation-Based Review of Classical, Heuristic, and Metaheuristic Path Planning Algorithms,” *Scientific Reports*, vol. 15, article 12643, 2025. DOI: https://doi.org/10.1038/s41598-025-96614-2

[5] V. Nur’aini and F. Haryani, “The Creation of the SnaPhys (Snake Physics) Game as a Learning Medium for Physics on Renewable Energy Topics,” *Journal of Advances in Physics Education*, vol. 1, no. 1, pp. 13–24, 2025. DOI: https://doi.org/10.23960/jape.v1i1.364